Overall Instructions: Due by Friday, Oct. 30th, 11:00pm EST. Please submit a .pdf of your answers via Canvas, including a url to your .ipynb notebook in a public GitHub repository. Use the provided LaTeX template for your submission. Please name your Canvas submission as Lastname.pdf, (*e.g.* Trujillo.pdf). If you need help with LaTeX or GitHub, please visit during office hours or post questions to the Canvas. Include your code in a (well-documented, commented, clear) `assignment02.ipynb` file. Scores for the code-based questions below will be based on your written responses *and* code.

Problem 1 (20 points)

In this question we will work with degree-controlled random graph null models.

(a) The Configuration Model (5 points).

First, implement the configuration model in Python. Your function should take a list of integers as a degree sequence, and return a graph where each vertex has the assigned degree. You graph may contain duplicate edges and self-loops.

(b) The Chung Lu Model (5 points).

Next, implement the Chung Lu random graph model. Remember that this model creates an edge between nodes i and j with independent probabilities defined by vertex degree and the total number of edges in the graph:

$$p_{ij} = \frac{k_i k_j}{2m},$$

where $2m := \sum_i k_i$.

Your implementation should take a list of integers as a degree sequence, and return a *simple* graph with approximately the desired degree sequence.

(c) Model Differences (5 points).

For the degree sequence `degrees=[8,5,5,4,3,2,2,2,1,1,1,1,1,1,1]`, generate a thousand random graphs using the Configuration Model and the Chung Lu model, then:

(i) Plot the average adjacency matrix for each model. This should be a heatmap of:

$$\langle \boldsymbol{A} \rangle = \frac{1}{N} \sum_{i=1}^{N} \boldsymbol{A}_i$$

where N is the number of generated random graphs, and $\boldsymbol{A}_i$ is the adjacency matrix of the ith graph. *(Hint: matplotlib's `imshow` is useful for displaying matrix data.)*

(ii) Choose two graph metrics: one that should be approximately the same across graphs generated by both models, and one that should diverge significantly between the two models. For each metric, make a plot (histogram, kernel-density estimate, or cumulative distribution function) showing the distribution of the metric across all generated graphs from each model.

Note: For this stage you may remove self-loops and duplicate edges from your configuration model graphs if it helps make a direct comparison to Chung Lu graphs.

(d) Discussion (5 points).

Discuss the above results briefly. Why do the models produce similar results under some metrics and different results under others?

Solution:

Your solution here...

Problem 2 (20 points)

In this question we will gain experience with community detection.

(a) Finding communities (5 points).

Download a network dataset from https://networkrepository.com/ with between roughly $1,000 < N < 5,000$ nodes (select a different dataset from the one you used in assignment 1). Select a modularity maximization algorithm to detect communities in this graph.

(i) Visualize your network, coloring the nodes by community membership.

(ii) Report the modularity of the partition your algorithm found.

(iii) Report the number of communities detected.

(iv) Store your partition as a dictionary in the form of {`node_id: community_id`}.

(b) Null models (5 points).

Create a degree-preserving randomization of your network, and run the community detection algorithm you chose in part a on your randomized graph. Store each partition as a dictionary in the form of {`node_id: community_id`}.

(c) Plotting a null model comparison (5 points).

You should now have two dictionaries, corresponding to the partitions from parts a and b. Create a figure with two subplots: In each, plot the community size distribution of each partition. Your figure should be publication-quality, with well-labeled, consistent axes, log-scaled wherever relevant, etc.

(d) Analysis (5 points).

For the different partitions computed above, discuss how you would approach comparing this functions quantitatively. For example, imagine a function $f(p_1, p_2)$ that takes partitions p_1 and p_2 as input and returns a number corresponding to how similar or dissimilar the partitions are. What properties should this function f have? For this question, base your answer on a technique used in the literature, after a brief search about methods for quantifying differences between graph partitions. You do not need to implement the function, a high-level discussion is sufficient.

Solution:

Your solution here...

Problem 3 (20 points)
In this question we will revisit degree sequences with a focus on preferential attachment.

We have worked with $G(n, p)$ random graphs extensively. We have also discussed how most graphs are *not* random, but follow some kind of preference for homophily or highly "desirable" nodes. One common preference is to connect to high *degree* nodes, reinforcing a rich-get-richer dynamic. We will create a *network growth model*, where we add new vertices to a graph, and create edges to existing vertices with a probability based on vertex i's degree k_i:

$$p(i) = \frac{k_i^\alpha + 1}{\sum_{j=0}^{n}(k_j^\alpha + 1)}$$

The α parameter allows us to configure the model between a sublinear preference for high-degree nodes ($0 < \alpha < 1$), a linear preference ($\alpha = 1$), and a superlinear preference ($\alpha > 1$). The $+1$ in the above equation ensures some chance of connecting to any vertex, even if it initially has a degree of zero.

(i) Implement the generator (4 points).

Write a graph generating function `preferentialGenerator(n, a)` that returns a graph with n vertices and $\alpha = a$.

(ii) Degree analysis (4 points).

Generate graphs with 1,000 nodes and alpha values of 0.1, 1, and 2. Make a plot comparing the three degree distributions. You may use histograms, kernel-density estimates, or cumulative distribution functions. However, your plots should be easy to interpret: select number of bins or bin widths, log scale, or other attributes as needed to produce a high-quality figure.

We will now pivot to the *random walk* model of network generation. In this model, every time we add a vertex i we add two edges. First, an edge to an existing random vertex j. Then, with probability p, we add an edge from i to a random neighbor of j. Otherwise we instead add an edge to another random vertex in the graph.

(iii) Implement the generator (4 points).

Write a graph generating function `randomWalkGenerator(n, p)` that returns a graph with n vertices generated by random walks with probability p.

(iv) Clustering analysis (4 points).

Generate graphs with 1,000 nodes and p values of 0, 0.1, 0.5, and 0.9. Make a plot comparing the four clustering distributions. As in part (ii), make these plots easily interpretable.

And finally...

(v) Interpretation (4 points).

Compare the graphs created by the two network growth models. What kind of structures do they produce? In what circumstances might we want to use preferentialGenerator as a null model, and when might the random walk model be a more appropriate null assumption?

Solution:

Your solution here...

Problem 4 (20 points)

We are training you to conduct original research, and a crucial component of that is a *literature review*. We can conduct literature reviews for several reasons, including familiarizing ourselves with a topic of study, a set of related methods, or the conventional wisdom of a field. When beginning a research project we typically want to know whether our research question has been asked before (and if so, using what tools and with what data), what related questions have been asked, and what gaps in understanding remain that we can try to answer.

Today we will start small. Search Google Scholar for five papers on different community detection algorithms. Don't use the ones we've discussed extensively in class (label propagation, Louvain, Girvan-Newman). For each algorithm, answer the following:

(a) What measure is being optimized (how do we know a given partition is optimal or good? Are they use modularity maximization, or something else?)

(b) Algorithmically (a high level explanation in a few sentences is fine), how are partitions selected / sampled?

(c) When did this technique come out? Does the journal or conference suggest what field this algorithm is used in? Is it terribly obscure? (We can use citation count as a proxy for obscurity here)

(d) Is the algorithm particularly well-suited to certain kinds of graphs? (Does it assume sparse graphs? A core-periphery structure? Does it scale well / poorly? Add a little context about where you might / should not use this algorithm)

I encourage you to read selectively; you want to understand how these algorithms work and where and how they could be used, but you don't need to understand every check for division by zero or the full literature review of how this approach has been used in MRI analysis and how detected neuronal communities differ from established medical literature. Read papers for what you need from them.

Solution:

Your solution here...

Problem 5 OPTIONAL (5 bonus points)
For every problem set this semester, there will be an opportunity to earn extra credit. Since this is a new(-ish) course, we are eager to receive genuine feedback on the material, pace, instruction style, etc. of the course so far. Please provide any feedback about the lectures in recent weeks and/or this assignment.

Solution:

Your (optional) solution here...